

Differentiable RDycore, Step One:

Adjoint-Based Manning Roughness Calibration

A proposed first paper and development plan

Mark Adams (LBNL)
prepared for Gautam Bisht and the RDycore team

August 18, 2026

1 Summary

I propose we build a *discrete adjoint* capability into RDycore using PETSc’s TSAdjoint [1] and drive it to one concrete, publishable endpoint: an end-to-end system that reads surface observations (USGS stream gauges first), runs RDycore forward and adjoint solves inside a gradient-based optimization loop (PETSc/TAO), and outputs a **calibrated Manning roughness map** on the Houston 1 km Hurricane Harvey mesh. The adjoint returns the gradient of a gauge-misfit functional with respect to *every* cell’s roughness at the cost of roughly one extra forward solve, independent of the number of parameters — this is what makes distributed calibration tractable where trial-and-error and ensemble screening are not.

This is deliberately the smallest slice of the larger differentiable-RDycore plan (adjoint → calibration → PyTorch coupling → learned physics) that yields a standalone paper and a strong exhibit for next year’s program recompute.

Status addendum (August 18, evening): implemented and verified

The plan below has been executed. On branch `adams/manning-draft`, increments 0, 1, 1b, 2a–2d, 3, 4, 5, and 6 (twin) are implemented, finite-difference-verified, and committed, with 12 new CI tests (all green; full log: `plans/RESULTS-manning-draft.md`; paper draft: `papers/manning-calibration/`). Later sections are left in plan tense for reference.

Verified result	Value	Gate
Exact Roe flux Jacobian vs FD (wet jump / transcritical)	$7.6 \times 10^{-9} / 2.4 \times 10^{-10}$	$10^{-6}/10^{-5}$
Assembled matrix vs FD coloring, full matrix, 3 BC combos	$1.5\text{--}1.9 \times 10^{-8}$	10^{-6}
Adjoint dJ/du_0 and dJ/dn vs FD (RK4, BEULER, ARKIMEX)	$\sim 10^{-8} / \sim 3 \times 10^{-6}$	10^{-5}
Two-zone Manning recovery (TAO/BLMVM twin)	4.1×10^{-7}	2×10^{-2}
Per-cell recovery, 20 obs times + Tikhonov	8.0%	—
Houston Harvey map (2,746 cells, implicit, 300-it prelim.)	26.3%, map written	—

Highlights beyond the plan: the flux Jacobian is *exact* (a hand-written forward-mode differential of the flux code, entropy-fix branches included); **implicit stepping now works** — backward Euler and ARK-IMEX (friction implicit, per-cell solves), both with FD-gated adjoints — and was *required*: explicit friction NaNs at Houston’s $\Delta t = 30$ s exactly as Section 8 predicts. Discovered

along the way: a new explicit source treatment was prerequisite (the existing splittings embed Δt in the RHS); PETSc’s TSEULER has no adjoint; the ARKIMEX adjoint needs a zero explicit-part parameter Jacobian registered (else SEGV). Remaining: converged Houston run (in progress), real-gauge calibration — gauge and datum data are ready ([data/harvey-gauges/](https://data.harvey-gauges/)), blocked only on identifying the Houston mesh’s projected CRS (team question).

2 What exists and what is missing

The RDycore SWE solver today is a forward model: explicit finite-volume shallow water (Roe fluxes; HLLC planned but not yet implemented) with no Jacobian, no adjoint, and no gradient path to its parameters. There is a worked in-tree precedent to learn from — the heat development branch (not a baseline for this effort) carries an implicit time integrator, a Jacobian matrix, and `TSSetIJacobian` with analytic per-cell derivatives, including a libCEED Q-function Jacobian — but those Jacobians are all *diagonal* (local source terms). What is missing is precisely this plan’s contribution: the *coupled*, off-diagonal flux Jacobian on the cell-adjacency graph, and all adjoint/sensitivity machinery. Everything needed is either stock PETSc, demonstrated in-tree, or mechanical to add:

- Manning n is already a per-cell material property with per-region and per-domain public setters — the parameter infrastructure exists.
- The heat branch provides worked reference examples for Jacobian matrix creation, implicit-TS and IJacobian registration, and a CEED Q-function Jacobian; the SWE drag-source Jacobian is the structural twin of its source Jacobian (3×3 diagonal blocks instead of scalars).
- The SWE right-hand-side Jacobian is block-sparse (3×3 blocks on the cell-adjacency graph); Roe flux Jacobians are closed-form. The friction-source Jacobian is diagonal, and $\partial S / \partial n$ has just two nonzeros per cell.
- `TSAdjoint` differentiates PETSc’s own time integrators (including our explicit RK) with optimal trajectory checkpointing; we supply only the two Jacobian callbacks and a cost integrand.
- Gauge-site plumbing is native: the YAML `observations.sites` convention and its natural-ordering scatter (`src/time_series.c`) already select cells to observe; the calibration work adds only observed-*values* input and the misfit.
- An ARK-IMEX path is half-built: `TEMPORAL_ARK_IMEX` and a “friction on the LHS” source mode exist in the config schema and in the CEED source kernels — only the integrator wiring and the source Jacobian are missing (Section 8).
- The Houston 1 km Harvey case is in the repository, and the USGS Harvey gauge data is downloaded (746 stage/discharge gauges with locations, HydroShare DOI 10.4211/hs.c037167e497546a1bc1508dfb3 CC-BY; the remaining QC item is the stage-datum conversion). The LETKF twin-experiment driver provides the observation plumbing and the driver pattern to copy.

The work is CPU-backend only for the SWE operator — the heat branch’s example shows CEED can express *pointwise/diagonal* Jacobians as Q-functions (a viable later route for a GPU drag-source Jacobian), but there is no CEED path to the coupled off-diagonal flux-Jacobian assembly the adjoint needs, so the CEED SWE kernels remain forward-only and untouched. It lands behind a config flag with zero impact on existing runs, and needs no new dependencies. Known technical risk: adjoints of SWE are noisy at wetting/drying fronts; we validate on fully wet cases first and treat wet/dry observations as future work.

3 Related work: this is proven ground, and timely

Adjoint Manning inversion is mature. Identification of distributed Manning coefficients by adjoint analysis goes back two decades [2], with twin-experiment validation on unstructured finite-volume SWE [3] and GPU-accelerated adjoint optimizers for 2D SWE [4]. The method itself is not research risk.

Two production-grade analogs exist — neither on our terrain. DassFlow (INSA Toulouse) is a 2D finite-volume shallow-water code built for variational data assimilation, inferring Manning, bathymetry, and discharge, including from SWOT satellite altimetry [5, 6]. Thetis (Firedrake) calibrated a spatially varying Manning field for the entire Baltic Sea with an automatically generated adjoint [7], after storm-surge sensitivity studies that carefully treated the overfitting problem for distributed friction [8] and an extension to coupled morphodynamics [9]. Both validate our approach — and neither runs on DOE exascale machines or couples to an Earth system model.

The differentiable-solver wave is cresting on exactly our use case. A 2026 paper, Inunda [10], rewrote a simplified (local-inertial, raster) flood solver entirely in PyTorch and calibrated roughness by gradient descent against gauges — validated on a *Hurricane Harvey hindcast in Harris County* (0.67 m MAE against high-water marks). Hydrograd [11] (Julia) couples a 2D SWE solver with neural flow-resistance terms; AegirJAX [12] does differentiable coastal hydrodynamics in JAX; CaMa-Flood-GPU [13] names gradient-based Manning calibration as its goal; and differentiable rainfall-runoff models already infer Manning n with embedded neural networks [14].

Our differentiator. Every code in that last group *rewrote* a reduced solver inside an ML framework to obtain differentiability; the FE-framework codes cannot reach DOE leadership machines. RDycore + TSAdjoint is, to my knowledge, the only path that makes an existing, validated (471M cells), GPU-capable, E3SM-coupled DOE production code differentiable *without rewriting it*. Inunda’s Harvey result is the perfect foil: our Phase-3 demo is the same named U.S. flood event, in a code that also runs coupled continental-scale simulations on Frontier.

4 Incremental plan to the Manning map

Each increment is small, independently testable (CTest against finite-difference ground truth throughout), and lands behind a default-off config flag. Detailed version in `plans/manning-map-incremental-plan.md`.

#	Increment	Deliverable	RDycore-team input
0	Config + FD baseline	TS Set RHS Jacobian wired with PETSc FD-coloring Jacobian; flag default off	YAML schema approval
1	Source Jacobians	Analytic drag/bed-slope diagonal blocks, preallocated bs=3 matrix	Physics review ($h_{\min}$ floor, semi-implicit friction)
2a–d	Roe flux Jacobian	Split by risk: edge-level derivative harness; physical-flux + frozen-dissipation Jacobian (validates assembly on smooth flows); exact dissipation derivatives (the one hard item, with the harness as oracle and the frozen Jacobian as schedule offramp); BC blocks	Entropy-fix review; Roe vs HLLC decision
3	State adjoint	Trajectory + gauge-misfit cost; dJ/du_0 vs FD; new adjoint driver	Observation conventions, gauge file format
4	Parameter gradient	dJ/dn per region vs FD; first <i>sensitivity map</i>	Public API review (E3SM-facing)
5	TAO loop (per region)	Twin experiment recovers two-zone n	Bounds, test-case choice
6	Per-cell n + regularization	Calibrated Manning map on Houston1km; optional real gauges	Harvey gauge data ; regularization prior; acceptance criteria
7	Paper	Methods + twin + Harvey results	Co-authors, venue, scope

Rough total: ~1500 lines of new C (one library source file, one driver, a small API surface, five CTests). Increments 4–6 each produce a citable artifact (sensitivity map $\rightarrow$ recovered zones $\rightarrow$ calibrated map).

5 Strategic decisions — application-science lead

These are the choices that shape the paper and the recompute story, and they belong to the RDycore/application side, not the solver side:

1. **Observation strategy for paper 1.** Synthetic-only twin experiment vs. real USGS Harvey stage data (my recommendation: twin for validation, real gauges for the headline result if the data QC is manageable). The literature notes velocity observations constrain n more strongly than stage but converge less readily [3] — do we have or want any velocity data? Satellite: SWOT water-surface elevation is proven usable in this exact inversion [5] and is a natural second paper; flood-extent imagery interacts badly with wet/dry nondifferentiability and should stay future work.
2. **Roughness parameterization and prior.** Per-region (well-posed, few parameters) vs. per-cell with regularization (the interesting map, but overfitting is the known failure mode [8]). Choice of Tikhonov vs. total-variation, and whether the prior n_0 comes from land-cover data — that prior is application knowledge.
3. **Flux function on the adjoint path.** Roe is the committed path (the only Riemann solver implemented; its entropy-fix kinks are handled by wet-case validation and one-sided deriva-

tives). HLLC is the *backup*, triggered only if gradient validation or the Harvey calibration shows the kinks poisoning gradients — and it means implementing the forward HLLC flux first (RIEMANN_HLLC is enum-only today), then its Jacobian. Physics team owns the trigger call.

4. **Target case and wet/dry exposure.** Houston1km Harvey is the compelling demo but exercises wetting/drying; validation cases stay fully wet. How wet is the calibration window we pick?
5. **Public API shape.** What enters `rdycore.h` (`RDySetObservations`, `RDyAdjointSolve`, `RDyGetSensitivities`, ...) and whether Fortran/E3SM bindings are needed now or later.
6. **Venue and positioning.** WRR, JAMES, or GMD; and whether we engage Inunda [10] head-on with the same Harvey event (direct comparison, higher impact, more work) or cite and differentiate.
7. **Recompete framing.** How much of the PyTorch-bridge/AI-DA arc appears in paper 1’s outlook section versus being held for the proposal narrative.

6 Future work (enabled, not included)

The calibrated-map system is the substrate for: zero-copy PyTorch coupling via the existing PETSc $\leftrightarrow$ PyTorch bridge (`RDycore` as a `torch.autograd.Function`); neural source terms trained through the solve (model-error correction, surface-groundwater exchange); hybrid ensemble-variational DA with the LETKF line; SWOT and velocity observations; second-order adjoints for uncertainty on the recovered map; optimal gauge placement; implicit/IMEX stepping of the SWE system — IMEX with implicit friction is a near-term easy win quantified in Section 8, and fully implicit stepping on the new coupled Jacobian follows; and sediment-parameter estimation. Each is a proposal work package with the first paper as its demonstrated foundation.

7 Appendix: equations, algorithms, and the PETSc API they map to

Forward model (exists today)

`RDycore` solves the 2D shallow water equations in conservative variables $u = (h, q_x, q_y)$ with $\mathbf{q} = (q_x, q_y) = h\mathbf{v}$:

$$\partial_t h + \nabla \cdot \mathbf{q} = Q_r(x, t), \quad (1)$$

$$\partial_t \mathbf{q} + \nabla \cdot \left(\frac{\mathbf{q} \otimes \mathbf{q}}{h} + \frac{gh^2}{2} \mathbb{I} \right) = -gh \nabla z_b - g n^2 h^{-7/3} \mathbf{q} \|\mathbf{q}\|, \quad (2)$$

with rain/runoff forcing Q_r , bed elevation z_b , and Manning drag written in momentum form (equivalently $-C_D \|\mathbf{v}\| \mathbf{v}$ with $C_D = g n^2 h^{-1/3}$), exactly as coded in `src/swe/swe_petsc.c`. Cell-centered finite volumes on an unstructured DMPlex mesh give the semidiscrete system

$$\frac{du_i}{dt} = - \frac{1}{|\Omega_i|} \sum_{e \in \partial \Omega_i} \mathcal{F}(u_i, u_j, \mathbf{n}_e) L_e + S(u_i; n_i, t) \equiv f_i(u, p), \quad (3)$$

where $\mathcal{F}$ is the Roe approximate Riemann flux (the only Riemann solver implemented today; an upwinded variant exists, HLLC does not), with well-balancing and an $h_{\min}$ wet/dry floor. Time integration is explicit: forward Euler or explicit Runge–Kutta.

PETSc realization: mesh and state via `DMPlex`; the RHS $f(u, p)$ registered with `TSSetRHSFunction(ts, rhs, OperatorRHSFunction, rdy)` (`src/rdysetup.c`); integrator via `TSSetType (TSEULER/TSRK)`; windowed advance via `TSSetMaxTime/TSSolve`.

State Jacobian (increments 0–2)

Each interior edge $e = (i, j)$ contributes two 3×3 blocks per cell row, so the RHS Jacobian is block-sparse on the cell-adjacency graph:

$$J = \frac{\partial f}{\partial u}, \quad J_{ii} += -\frac{L_e}{|\Omega_i|} \frac{\partial \mathcal{F}}{\partial u_L} + \frac{\partial S_i}{\partial u_i}, \quad J_{ij} = -\frac{L_e}{|\Omega_i|} \frac{\partial \mathcal{F}}{\partial u_R}, \quad (4)$$

with closed-form Roe flux Jacobians $\partial \mathcal{F} / \partial u_{L,R}$ and the local drag/bed-slope block $\partial S / \partial u$ (from Eq. 2; note $\partial(h^{-7/3}) / \partial h$ terms are regularized by the same $h_{\min}$ floor as the forward model). Boundary conditions modify diagonal blocks only.

PETSc realization: matrix from `DMCreateMatrix` with adjacency preallocation; blocked insertion via `MatSetValuesBlocked` (`bs=3`, `MATBAIJ` on CPU, `MATAIJ` for device compatibility); registered with `TSSetRHSJacobian`; verified against finite differences with `SNESComputeJacobianDefaultColor` (`MatFDColoring`).

Cost functional, trajectory, and discrete adjoint (increments 3–4)

The gauge misfit over observation times t_k with sparse observation operator H (one row per gauge, selecting h at the gauge cell) is

$$J(p) = \frac{1}{2} \sum_k (Hu(t_k) - y_k)^\top R^{-1} (Hu(t_k) - y_k), \quad (5)$$

where y_k is the vector of observed stages at time t_k and R is the observation-error covariance — diagonal in practice, $R = \text{diag}(\sigma_g^2)$ with σ_g the error standard deviation of gauge g , so the misfit is the Gaussian negative log-likelihood and the calibrated n a maximum-likelihood estimate. The cost is optionally written as an integral $\int_0^T r(u, t) dt$ for the quadrature formulation.

In data-assimilation terms, Eqs. (5) and (8) together constitute *strong-constraint 4D-Var* with the control vector dominated by parameters rather than the initial state (the regularization term of Eq. (8) playing the role of the background term): observations distributed over a window, the non-linear model as a hard constraint, and the gradient from one adjoint sweep — the same formulation DassFlow [6] calls variational data assimilation. This is also the precise sense in which the adjoint line *complements* the LETKF line: both minimize the same Gaussian misfit, with ensembles as the natural minimizer for sequential state correction and adjoints for whole-window parameter estimation, which is what makes the hybrid EnVar step in Section 6 well-posed. `TSAdjoint` differentiates the *discrete* time stepper, so the backward sweep

$$\dot{\lambda} = -\left(\frac{\partial f}{\partial u}\right)^\top \lambda - \left(\frac{\partial r}{\partial u}\right)^\top, \quad \dot{\mu} = -\left(\frac{\partial f}{\partial p}\right)^\top \lambda, \quad (6)$$

(run in its RK-discrete form) returns $dJ/du_0 = \lambda(0)$ and $dJ/dp = \mu(0)$ exact to roundoff for the computed trajectory — the right property for FD validation and optimization. The parameter Jacobian for Manning n is two nonzeros per cell, from Eq. (2):

$$\frac{\partial S_{\mathbf{q}_i}}{\partial n_i} = -2g n_i h_i^{-7/3} \mathbf{q}_i \|\mathbf{q}_i\|. \quad (7)$$

PETSc realization: checkpointing via `TSSetSaveTrajectory` (`TSTrajectory`; memory at test scale, disk at Harvey scale); cost integrand via `TSCreateQuadratureTS`; Eq. (7) via `TSSetRHSJacobianP`; sensitivities registered with `TSSetCostGradients`(λ, μ) and computed by `TSAdjointSolve`; H as a sparse `MATAIJ` (`MatCreateAIJ`), with gauge cells specified through the existing `observations.sites` YAML convention (`RDyObservationSites`) and the LETKF driver’s observation matrix as the pattern.

Bound-constrained calibration (increments 5–6)

$$\min_{n_{lo} \leq n \leq n_{hi}} J(n) + \frac{\beta}{2} \|L(n - n_0)\|^2, \quad (8)$$

per-region n first (no regularization needed), then per-cell with $L = \mathbb{I}$ (Tikhonov) or a discrete-gradient L (smoothing/TV-like), prior n_0 from land cover.

PETSc realization: `TaoCreate` with `TAOBLMVM` (or `TAOBNCG`); bounds via `TaoSetVariableBounds`; the objective/gradient callback runs one `TSSolve` + `TSAdjointSolve` pair per iteration (gradient cost $\approx$ one forward solve, independent of the number of parameters); `TaoSolve` drives the loop.

Summary map

Component	Equations	PETSc API	Increment
Forward RHS $f(u, p)$	(1)–(3)	<code>TSSetRHSFunction</code> , <code>DMPlex</code>	exists
Explicit stepping	—	<code>TSSetType</code> (<code>TSEULER/TSRK</code>), <code>TSSolve</code>	exists
State Jacobian $\partial f / \partial u$	(4)	<code>TSSetRHSJacobian</code> , <code>MatSetValuesBlocked</code>	0–2
IMEX friction (Sec. 8)	(10)	<code>TSARKIMEX</code> , <code>TSSetIFunction/IJacobian</code>	1b
FD verification	—	<code>SNESComputeJacobianDefaultColor</code>	1–2
Trajectory checkpointing	—	<code>TSSetSaveTrajectory</code>	3
Cost functional	(5)	<code>TSCreateQuadratureTS</code>	3
Observation operator H	(5)	<code>MatCreateAIJ</code>	3
Adjoint sweep λ, μ	(6)	<code>TSSetCostGradients</code> , <code>TSAdjointSolve</code>	3–4
Parameter Jacobian $\partial f / \partial n$	(7)	<code>TSSetRHSJacobianP</code>	4
Bounded optimization	(8)	<code>TAOBLMVM</code> , <code>TaoSetVariableBounds</code> , <code>TaoSolve</code>	5–6

8 Implicit time stepping: the IMEX-friction opportunity

The Phase-1 source Jacobian enables a cheap, high-value integrator upgrade that is worth calling out separately: `TSARKIMEX` with the Manning drag term of Eq. (2) treated implicitly and the hyperbolic fluxes kept explicit. Because the drag is per-cell local, the implicit stage matrix $I - a \Delta t \partial S / \partial u$ is *block-diagonal* with 3×3 blocks (effectively 2×2 : drag touches only the momentum rows and that cell’s own h). The “solve” is an independent few-iteration Newton in every cell — no global system, no KSP, nothing to precondition, and trivially GPU parallel. It requires only the increment-1 source Jacobian, none of the flux blocks, and it replaces the current hand-rolled semi-implicit friction splittings (`SEMI_IMPLICIT` and the Xia–Liang-style `IMPLICIT_XQ2018`) with a principled, second-order, adaptive (`TSadapt`), and adjoint-compatible integrator.

Notably, this path is *already half-built* on `main`: `TEMPORAL_ARK_IMEX` and `SOURCE_ARK_IMEX` (“bed friction terms moved to LHS”) exist in the config schema with validation, and the CEED source kernels already branch on the IMEX source mode — but no `TSARKIMEX` is ever created and no `IFunction/IJacobian` is registered. Completing it is exactly the increment-1 source Jacobian

plus integrator wiring. A related fact strengthens the case: RDycore’s existing adaptive stepping (`target_courant_number` in the Harvey adaptive config) targets the CFL limit *only* — in friction-bound thin-water regimes that controller is flying blind, and implicit friction is what makes aggressive CFL-targeted adaptivity safe.

How much larger a step? A stability estimate

Explicit stepping faces two limits. The gravity-wave CFL condition, which IMEX-friction retains,

$$\Delta t_{\text{CFL}} \approx \frac{C \Delta x}{\sqrt{gh} + |v|}, \quad (9)$$

and the friction-stiffness limit, which it removes: linearizing the drag as $\dot{\mathbf{q}} = -\lambda \mathbf{q}$ with $\lambda = gn^2 h^{-4/3} |v|$ gives explicit stability only for

$$\Delta t \lesssim 2\tau_f, \quad \tau_f = \frac{h^{4/3}}{g n^2 |v|}. \quad (10)$$

The IMEX step-size gain in a cell is the ratio $\Delta t_{\text{CFL}}/2\tau_f$; setting it to one gives a *binding-depth threshold*: friction is the limiter wherever

$$h^{4/3} \lesssim \frac{\Delta x g n^2 |v|}{2C (\sqrt{gh} + |v|)}. \quad (11)$$

For the Houston 1 km mesh ($n = 0.015$ in the current config, $|v| \sim 0.1$ m/s) the threshold is $h \approx 5$ cm — and a rainfall-driven Harvey domain spends much of the event with large areas below it. Since the global explicit Δt is the minimum over all cells, a single thin wet cell drags the whole domain down to seconds; with implicit friction the domain runs at the CFL of its deepest/fastest cells.

Regime	$h, v $	Friction limit $2\tau_f$	CFL at $\Delta x=1$ km	IMEX gain
Dam break, post-break (ex2b)	5 m, 3 m/s	~ 40 min	CFL-bound	$1\times$
Bayou in flood	5 m, 2 m/s	~ 20 min	~ 110 s	$1\times$
Sheet flow	2 cm, 0.1 m/s	~ 35 s	~ 2000 s	$\sim 50\times$
Thin film / recession	5 mm, 0.1 m/s	~ 8 s	~ 4000 s	$\sim 500\times$
Drying front	1 mm, 0.05 m/s	~ 2 s	$\sim 10^4$ s	$\sim 10^3\times$ (capped by $h_{\min}$)

Two cases anchor the estimate. In the *deep* dam-break test (ex2b: 10 m over 5 m, $n = 0.015$), $\tau_f \approx 20$ minutes while the CFL step is a fraction of a second — friction is four orders of magnitude from binding, so IMEX buys *robustness, not speed*, in that phase. On the Houston Harvey case the picture inverts after peak flood: the long recession/drainage tail that dominates wall-clock is exactly where the drag is stiff, and the achievable stable step improves by roughly **one to two orders of magnitude** (approaching three at drying fronts, where the $h_{\min}$ floor caps the estimate).

Why not just keep the semi-implicit splitting? The adjoint argument

For the *forward* model, the existing splittings already bank most of the step-size gain in the table, so the raw-speed argument for IMEX is modest. The decisive argument is structural: the semi-implicit treatments embed Δt *inside the RHS callback* (they reconstruct the end-of-step momentum algebraically within f), so what the integrator sees is not an ODE right-hand side — it is consistent only

with forward Euler, and it admits no well-defined $\partial f/\partial u$ to give to `TSSetRHSJacobian`. `TSAdjoint`'s guarantee — exact derivatives of the discrete stepper — assumes a genuine f . That leaves exactly two clean options on the adjoint path: (a) plain *explicit* friction, which is fine for the wet validation cases (increments 0–5) but collapses Δt to seconds in the thin-water phases of a Harvey window, inflating adjoint cost and trajectory storage by the table's 10–100 $\times$; or (b) friction moved into `TSSetIFunction` with its block-diagonal `TSSetIJacobian` under `TSARKIMEX`, which `TSAdjoint` differentiates natively. **We adopt (b)**: increment 1b completes the started `SOURCE_ARK_IMEX` path and must land before the increment-6 Harvey calibration; it is the enabling step for the paper's headline result, not a performance side quest. A second-order benefit: at large steps, first-order splitting error would be absorbed into the calibrated n as discretization bias — the IMEX scheme keeps the recovered roughness answering for physics, not for the splitting.

Two remaining qualifications. (i) The gain *grows with calibration*: $n = 0.015$ is “smooth,” and realistic calibrated urban/vegetated values (0.03–0.1) raise gn^2 by 4–40 $\times$, pushing the binding depth of Eq. (11) toward tens of centimeters — the Manning maps this plan produces make the IMEX path more valuable, not less. (ii) These are frozen-coefficient linear-stability estimates, good to a factor of ~ 2 ; the true numbers fall out for free once `TSARKIMEX` runs with `TSAdapt` reporting its own steps.

References

- [1] Hong Zhang, Emil M. Constantinescu, and Barry F. Smith. PETSc TSAdjoint: A discrete adjoint ODE solver for first-order and second-order sensitivity analysis. *SIAM Journal on Scientific Computing*, 44(1):C1–C24, 2022.
- [2] Yan Ding, Yafei Jia, and Sam S. Y. Wang. Identification of Manning's roughness coefficients in shallow water flows. *Journal of Hydraulic Engineering*, 130(6):501–510, 2004.
- [3] Xianqiang Chen et al. Adjoint model of shallow water equations based on the finite volume method and unstructured meshes. *Abstract and Applied Analysis*, 2014:407468, 2014. Twin-experiment inversion of Manning roughness; velocity vs. stage observation sensitivity.
- [4] Asier Lacasta, Mario Morales-Hernández, Pilar García-Navarro, et al. A GPU accelerated adjoint-based optimizer for inverse modeling of the two-dimensional shallow water equations. *Computers & Fluids*, 2016.
- [5] Pierre Brisset, Jérôme Monnier, Pierre-André Garambois, and Hélène Roux. River discharge estimation from synthetic SWOT-type observations using variational data assimilation and the full Saint-Venant hydraulic model. *Journal of Hydrology*, 558:176–195, 2018.
- [6] Jérôme Monnier et al. DassFlow: Data assimilation for free-surface flows (DassHydro platform). <https://dasshydro.github.io>, 2025. 2D finite-volume shallow water with variational DA; Manning, bathymetry, and discharge inference incl. SWOT/AirSWOT and video-derived surface velocity.
- [7] Tuomas Kärnä et al. Efficient optimization of a regional water elevation model with an automatically generated adjoint. *Journal of Advances in Modeling Earth Systems*, 15, 2023. Spatially varying Manning friction calibrated for the Baltic Sea with Thetis/Firedrake/pyadjoint.
- [8] Simon C. Warder, Kevin J. Horsburgh, and Matthew D. Piggott. Adjoint-based sensitivity analysis for a numerical storm surge model. *Ocean Modelling*, 160:101766, 2021.

- [9] Mariana C. A. Clare et al. Calibration, inversion and sensitivity analysis for hydro-morphodynamic models through the application of adjoint methods. *Computers & Geosciences*, 163:105104, 2022.
- [10] Zhi Li. Inunda: A GPU-native, agent-enabled, differentiable solver for high-resolution flood inundation modeling. arXiv:2607.09614, 2026. PyTorch local-inertial solver; roughness/bed/infiltration by gradient descent; Hurricane Harvey hindcast, Harris County TX.
- [11] Xiaofeng Liu et al. Scientific machine learning of flow resistance using universal shallow water equations with differentiable programming. *Water Resources Research*, 61, 2025. Hydrograd (Julia/SciML): AD-based Manning inversion and neural flow-resistance terms.
- [12] Elsa Cardoso-Bihlo and Alex Bihlo. A solver-in-the-loop framework for end-to-end differentiable coastal hydrodynamics. arXiv:2604.07129, 2026. AegirJAX: differentiable non-hydrostatic shallow-water solver in JAX.
- [13] Shengyu Kang, Jiabo Yin, and Dai Yamazaki. CaMa-Flood-GPU: A GPU-based hydrodynamic model implementation for scalable global simulations. *Geoscientific Model Development*, 19:5623–5640, 2026.
- [14] Tadd Bindas et al. Improving river routing using a differentiable Muskingum-Cunge model and physics-informed machine learning. *Water Resources Research*, 60, 2024. Differentiable routing inferring Manning roughness and channel geometry with neural networks.